

Technical Reflection Report

Global Logistics Management System (GLMS) — PROG7311 POE Part 3
ST10435542 — Natheer Jardien

1. Introduction

In Part 3 of this Portfolio of Evidence the GLMS prototype was modernised from a single monolithic MVC application into a Service-Oriented Architecture. The business logic and all database access now live inside an ASP.NET Core Web API (the only component that talks to SQL Server), the MVC application was refactored into a pure presentation layer that consumes the API over HTTP with JWT authentication, and the whole ecosystem — database, API and web frontend — is containerised with Docker and orchestrated through a single docker-compose.yml file. This report reflects on the two engineering practices that make this architecture production-ready: automated testing inside a CI/CD pipeline, and containerisation.

2. DevOps and Testing: why automated testing is critical in a CI/CD pipeline

Continuous Integration/Continuous Delivery (CI/CD) is built on one idea: every change to the code is integrated, built and verified automatically, so that the software is always in a releasable state (Fowler, 2013). The verification step is only trustworthy if it is automated. A human tester cannot re-test an entire enterprise system every time a developer pushes a commit; an automated test suite can, in minutes, and it applies exactly the same checks every single time.

In the GLMS solution this is not theoretical. The test project contains two layers of automated protection. The unit tests verify the business logic in isolation — for example that the USD-to-ZAR currency calculation multiplies correctly against a mocked exchange rate, that the State pattern refuses illegal contract transitions (an Expired contract can never be re-activated), and that the file validator throws a security exception for .exe uploads. The integration tests then boot the real Web API in memory and fire genuine HTTP requests at it: they assert that GET /api/contracts returns 200 OK with valid JSON, that an unauthenticated request is rejected with 401, that a contract created through POST can be read back with identical data, and that the API answers 400 Bad Request when a service request targets an Expired contract.

These tests run automatically in GitHub Actions on every push and pull request. That is precisely how automated testing prevents bugs from reaching production: a 'breaking change' — for instance a refactor that accidentally bypasses the contract status rule — fails the pipeline before the code can ever be merged or deployed. The bug is caught at the cheapest possible moment, while the developer still has the context in their head, instead of being discovered by a client after deployment (Humble and Farley, 2010). During this very project the integration suite caught a real defect: a model-binding issue that silently made POST /api/contracts ignore its JSON body. A manual click-through of the UI might have missed it; the automated Create-then-Read test failed instantly and pinpointed the problem.

3. Containerisation: how Docker solves “it works on my machine”

The classic failure mode of multi-environment software is drift: the developer's laptop runs one version of the .NET runtime and a local SQL Express instance, the test server runs another, and production runs something different again. Software that passed every check in development then fails in production for reasons that have nothing to do with the code — a missing dependency, a different OS library, a different connection string. This is the “it works on my machine” problem (Merkel, 2014).

Docker solves it by packaging the application together with its entire runtime environment into an image. The GLMS API image, for example, is built with a multi-stage Dockerfile: the SDK image compiles and publishes the app, and the output is copied into a slim ASP.NET runtime image. That image is immutable — the exact same bytes run on my Windows laptop, on a lecturer's machine, in a GitHub Actions runner, or on a Linux server in the cloud. Development, testing and production no longer share only the source code; they share the identical runtime artefact, so an image that passed the test stage is bit-for-bit the image that is deployed (Docker, 2026).

`docker-compose.yml` extends this consistency from one application to the whole ecosystem. One command, `docker compose up`, produces the same three-container topology anywhere: `sql-server-db` (SQL Server 2022 with a persistent volume), `glms-backend-api` (which waits for the database healthcheck, applies EF Core migrations automatically and seeds the default logins) and `glms-frontend-web`. The containers communicate over an internal bridge network using service names — the API's connection string points at “`sql-server-db`” and the frontend calls “`http://glms-backend-api:8080/`” — and all environment-specific values (connection string, JWT signing key, API base URL) are injected as environment variables rather than baked into code. Nobody has to install SQL Server, trust a development certificate or edit `appsettings.json` to run the system; the environment itself is version-controlled alongside the code.

4. Conclusion

Together the two practices are complementary halves of one safety net. Automated tests guarantee that the behaviour of the system is correct before deployment; containers guarantee that the environment the tests ran in is the environment production will use. With both in place, the GLMS can evolve — new endpoints, new business rules, horizontal scaling of the API container — with confidence that regressions are caught by the pipeline and that “it worked in the pipeline” genuinely means “it will work in production”.

Reference List

- Docker. (2026). *Docker overview*. [online] Available at:
<<https://docs.docker.com/get-started/docker-overview/>> [Accessed 11 July 2026].
- Fowler, M. (2013). *ContinuousDelivery*. [online] Available at:
<<https://martinfowler.com/bliki/ContinuousDelivery.html>> [Accessed 11 July 2026].
- Humble, J. and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston: Addison-Wesley.
- Merkel, D. (2014). Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).

Microsoft. (2026). *Integration tests in ASP.NET Core*. [online] Available at:
<<https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests>> [Accessed 11 July 2026].